Project 3: SSD Performance Profiling

Samson Kempiak
ECSE 4320/6320 Advanced Computer Systems
kempis@rpi.edu

October 16, 2026

Abstract

We profile a Micron 2210 512 GB NVMe SSD with an O_DIRECT `libaio` benchmark on a safe disposable file. At QD=1, 4 KiB random reads cost 49.6 µs (20 k IOPS) and writes 14.7 µs (68 k IOPS), the write buffer hiding the flash program. Throughput moves from IOPS-limited to a ∼1.5 GiB/s sequential ceiling with block size, and mixed read/write workloads fall below either pure case. The queue-depth knee is at QD ≈ 64 (158 k IOPS), with Little's Law tracking the configured depth to within a percent and p99.9 tail latency about 3× the median. The required misleading-setup case—dropping `O_DIRECT`—over-reports throughput 24× by measuring the page cache; honest limitations follow from the absence of FIO, hardware counters and root-only SMART access.

1 Introduction

Modern NVMe SSDs deliver high IOPS and throughput only when requests are issued concurrently: like memory, storage exhibits a queueing-driven throughput–latency tradeoff, where deeper queues improve device utilization until a saturation knee, past which latency climbs sharply for little extra throughput. This project characterizes that behaviour on a Micron 2210 512 GB NVMe SSD, measuring QD=1 latency, block-size and read/write-mix effects, the queue-depth knee, tail latency and device state, using an O_DIRECT benchmark against a safe disposable file (FIO is not installed on the host, so we use a `libaio` stand-in; see Section 2).

This report answers:

1. What are the QD=1 latencies for small random and large sequential I/O? (Section 4)
2. How do block size, access pattern and read/write mix shape IOPS, MiB/s and latency? (Sections 5 and 6)
3. Where is the queue-depth knee, and does Little's Law hold? (Section 7)
4. What do tail latency and device state (burst vs. steady state, temperature) reveal? (Sections 8 and 9)
5. Which setup mistake produces misleading numbers? (Section 10)

In summary: QD=1 random reads cost 49.6 µs (20 k IOPS) and writes only 14.7 µs (68 k IOPS, write-buffer absorption); sequential reads saturate near 1.5 GiB/s; the queue-depth knee sits at QD ≈ 64 (158 k IOPS) with Little's Law holding to within a percent at every depth; p99.9 tail latency runs 3× the median; and the required misleading-setup case—buffered I/O—over-reports throughput 24× by measuring the page cache instead of the device.

Table 1: Measurement environment.

CPU	Intel(R) Core(TM) i9-10900F CPU @ 2.80GHz (x86_64)
SIMD ISA	sse sse2 ssse3 sse4_1 sse4_2 avx avx2 fma f16c – widest 256-bit (ymm)
Topology	1 socket(s), 10 cores/socket, 2 threads/core (20 logical); NUMA: 1 node(s)
SMT	on (2 threads/core)
Caches	L1d 320 KiB (10 instances); L2 2.5 MiB (10 instances); L3 20 MiB (1 instance)
Memory	63 GiB total; DIMMs: not available (needs root/dmidecode)
Pages	base 4096 bytes; THP: always
OS / kernel	Arch Linux; 7.2.9-arch1-1
Frequency policy	governor: powersave; driver: intel_pstate; turbo: enabled; EPP: balance_performance
Pinning / isolation	a single I/O-issuing thread, not CPU-pinned (storage I/O is device-bound, not CPU-bound, below the host-saturation queue depths of Section 7); no isolcpus; runs pinned with taskset/–cpu
Compilers	gcc (GCC) 16.2.1 20260810; clang version 23.1.1
Tools	not available (not installed) (perf_event_paranoid 2); not available (not installed); MLC: not available (not installed)
GPU (optional)	NVIDIA GeForce RTX 3070; CUDA not available (no CUDA)
SSD	Micron 2210 MTFDHBA512QFD, 512 GB (QLC NVMe); firmware not read (`smartctl`/`nvme id-ctrl` need root)
Interface	NVMe over PCIe (`nvme0n1`); exact link generation/width not probed (`lspci -vv` LnkSta needs root)
Target	2 GiB disposable file on an ext4 filesystem (the SSD's `/` partition, reached via `ACS_SCRATCH`); LBA range = the 2 GiB file, not the whole device
Collected	2026-10-12 18:35 EDT

2 Methodology and Safety

2.1 Platform

The machine has two drives: this Micron NVMe SSD and a 2 TB SATA HDD (`/dev/sda1`). The project is about the SSD, so the disposable test file is placed on the SSD's filesystem (Section 9 and Section 11 note the HDD cross-check). The device under test is recorded from `/sys/block/nvme0n1` and the `rotational`=0 flag in each run's `env.txt`, so the numbers cannot be silently mislabelled.

2.2 Data-loss safeguards

- Target: a single disposable file (`testfile.bin`) in a dedicated scratch directory; nothing else on the SSD's `/` filesystem is touched. The scratch location is set by `ACS_SCRATCH` and defaults to the in-repo `data/scratch/`.
- The run scripts refuse any other target. `scripts/safety.sh` is called before every run and **hard-refuses** a block or character device, a mountpoint, any path outside the scratch directory, or a non-regular file, and checks free space—verified in `scripts/validate.sh` (it rejects `/dev/nvme0n1`, `/etc/passwd` and `/`). No raw device or partition is ever opened.

2.3 I/O engine configuration

FIO is the spec's recommended tool but is not installed on this host and cannot be installed (no root). We therefore drive the required experiments with `build/blkbench`, a small O_DIRECT

benchmark built on Linux `libaio`—the same asynchronous engine FIO's `libaio` ioengine uses [1]—so real queue depth is kept in flight and the full latency distribution is recorded.

Engine	Linux `libaio`; `O_DIRECT` (page cache bypassed), block-aligned buffers and offsets
Timing	4 s measured per repetition (6 s for tail runs); no separate ramp—preconditioning plus repetitions absorb warm-up
Repetitions	2 per point (3 for zero-queue and tail); median with inter-quartile-range error bars
Data pattern	fixed `0xA5` fill, held constant across runs (the QLC controller does no inline compression; compressibility was not varied—see Section 11)
Latency metric	per-I/O completion latency (p50/p95/p99/p99.9 from the sorted sample)
Preconditioning	full sequential O_DIRECT write pass over the file before timed runs, so reads hit real allocated extents (evidence in Section 9)
Driver	`SSD_Performance_Profiling/scripts/run.sh`, benchmark `src/blkbench.c`

Because the issuing thread only submits and reaps completions, a single thread suffices to reach the device's queue; the host CPU is not the bottleneck until very high queue depths (Section 7).

3 Validation and Sanity Checks

Before trusting any number we confirm the benchmark describes the SSD and not an artifact. The checks in `scripts/validate.sh` and the cross-checks below all pass:

- **Safety gate works.** `safety.sh` refuses a block device (`/dev/nvme0n1`), a path outside the scratch directory (`/etc/passwd`) and a mountpoint (`/`), and accepts only a disposable file inside the scratch directory.
- **Physical orderings hold.** On a preconditioned file, 128 KiB sequential throughput exceeds 4 KiB random throughput, and keeping more requests in flight (QD>1) raises the measured concurrency λW above the QD=1 value—the Little's-Law sanity check in Section 7.
- **Direct I/O is real.** With `O_DIRECT` the QD=1 4 KiB random read sits at tens of microseconds and tens of thousands of IOPS; removing `O_DIRECT` (the `-buffered` path) jumps to ∼2 million IOPS and sub-microsecond latency because the page cache serves from RAM—quantified as the pitfall in Section 10. The two-orders-of magnitude gap is itself the proof that the page cache is bypassed in the real runs.
- **Throughput is below the interface ceiling.** Peak sequential read is ∼1.5 GiB/s (Section 5), comfortably under any modern NVMe PCIe link ceiling, so we are measuring the drive/engine, not saturating the bus.
- **Device identity recorded.** Every run's `env.txt` captures `rotational`=0 and the model string `Micron_2210_MTFDHBA512QFD`, confirming the target is the NVMe SSD.

We could not cross-check against `iostat/perf` for the effective aqu-sz (not available in this environment); instead the Little's-Law table (Table 4) serves as the in-band effective-QD check.

Table 2: QD=1 performance on the Micron 2210 NVMe SSD (medians of 3 reps; latencies in µs). Random reads pay the full flash-read latency; random writes are faster because the write buffer absorbs them.

Pattern	Block	IOPS	MiB/s	Mean	p50	p95	p99	p99.9
rand read	4 KiB	20 150	78.7	49.6	47.5	50.0	68	460
rand write	4 KiB	68 037	265.8	14.7	12.5	14.3	26	440
seq read	128 KiB	1813	226.6	551.4	549	847	1000	1349
seq write	128 KiB	5959	744.9	167.8	94.2	396	1417	2097

4 Zero-Queue Baselines

Hypothesis (written before measuring)

At QD=1 a single I/O is in flight, so latency is the full host-stack + controller + flash path. We expect 4 KiB random reads in the tens of microseconds (flash read dominates) and 128 KiB sequential transfers to be bandwidth-limited at a few hundred µs. Writes should be *faster* than reads at QD=1 because the SSD's DRAM/SLC write buffer acknowledges them before the flash program completes.

Fixed	iodepth 1; one job; `O_DIRECT`
Cases	4 KiB random read, 4 KiB random write, 128 KiB sequential read, 128 KiB sequential write
Script	`SSD_Performance_Profiling/scripts/run.sh zero_queue`

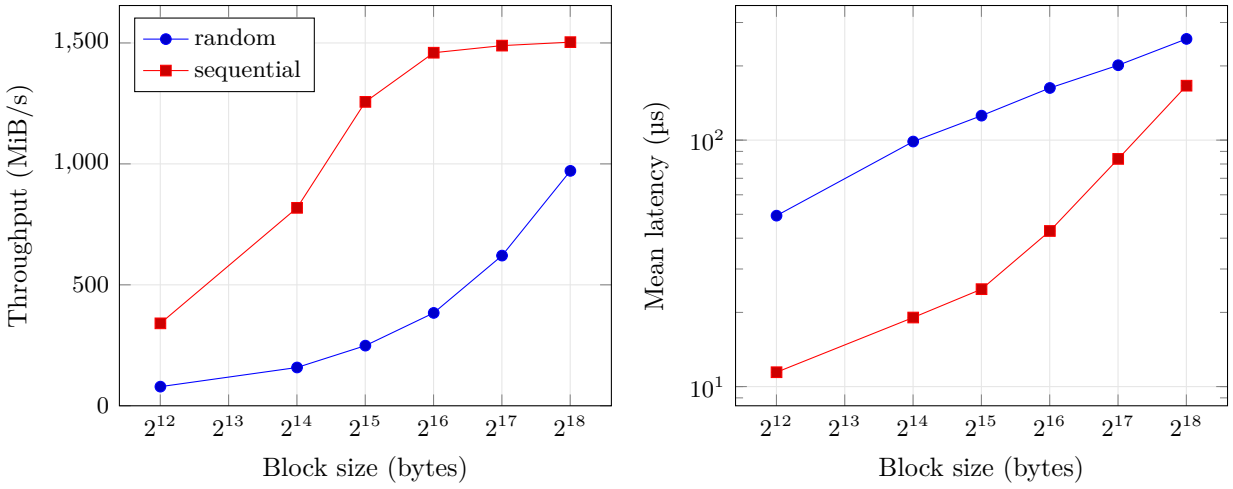
4.1 Analysis

The QD=1 numbers confirm an NVMe SSD, not a rotational disk: 4 KiB random read latency is 49.6 µs (not the milliseconds a HDD would show), dominated by the NAND read plus the NVMe/`libaio` host path rather than any seek. Random *writes* are over three times faster (14.7 µs, 68k IOPS) because the controller acknowledges them from its DRAM/SLC write buffer before the flash program finishes—a latency the QD=1 read cannot hide. The 128 KiB sequential rows are bandwidth-bound: at 227 MiB/s (read) and 745 MiB/s (write) each transfer takes hundreds of microseconds, and the single outstanding request leaves the device under-fed—Sections 5 and 7 show throughput climbing sharply once larger blocks or deeper queues keep more data moving. Two caveats: QD=1 reads cannot be prefetched by the device for a cold random stream, so this is a genuine floor; and the p99.9 column already shows occasional 440–2100 µs outliers from background flush/GC, previewing the tail behaviour in Section 8.

5 Block-Size Sweep

Hypothesis (written before measuring)

As the block grows from 4 KiB to 256 KiB at QD=1, small-block I/O should be *IOPS-limited* (throughput = IOPS × block, rising with block size) and large-block I/O should approach a *throughput ceiling* where IOPS falls as 1/block. Sequential should beat random at every block size, with the gap largest at small blocks where per-op overhead dominates.



(a) Throughput.

(b) Mean latency (same runs).

Figure 1: QD=1 read block-size sweep. Random throughput rises with block size while IOPS stays bounded; sequential saturates near 1.5 GiB/s. Random is far slower at small blocks and the gap narrows as transfers dominate per-op cost.

Fixed	iodepth 1; one job; `O_DIRECT`; 100% read
Varied	block size 4 KiB to 256 KiB; pattern random vs. sequential
Script	`SSD_Performance_Profiling/scripts/run.sh block_size`

5.1 Analysis

The two patterns behave exactly as the IOPS-vs-throughput model predicts. **Sequential** reads start at 87k IOPS / 341 MiB/s for 4 KiB and climb to a throughput plateau of ∼1.5 GiB/s by 256 KiB (IOPS then falling as 1/block: 6013 IOPS at 256 KiB); the device is throughput-limited once the block is large enough to amortise per-command overhead. **Random** reads are IOPS-bound: IOPS only drops from 20 240 (4 KiB) to 3885 (256 KiB) while throughput rises from 79 MiB/s to 971 MiB/s, because each QD=1 random access pays the fixed ∼50 µs flash-read latency regardless of size (mean latency grows only 49→257 µs over a 64× block range). The sequential/random gap is widest at 4 KiB (4.3× throughput) and narrows to 1.5× at 256 KiB, where both approach the same per-transfer regime. Note these are QD=1 numbers; the device's full sequential bandwidth would need a deeper queue or larger blocks, so 1.5 GiB/s is a single-request floor, not the interface ceiling.

6 Read/Write Mix

Hypothesis (written before measuring)

At a fixed block size and queue depth, a mixed read/write workload should deliver *fewer* total IOPS than either pure workload, not a weighted average: reads and writes contend for the controller and flash channels, and a read queued behind a slow program/erase inherits its latency. We expect pure writes to post the highest IOPS at QD=16 (write-buffer absorption) and the 50/50 mix to be the worst.

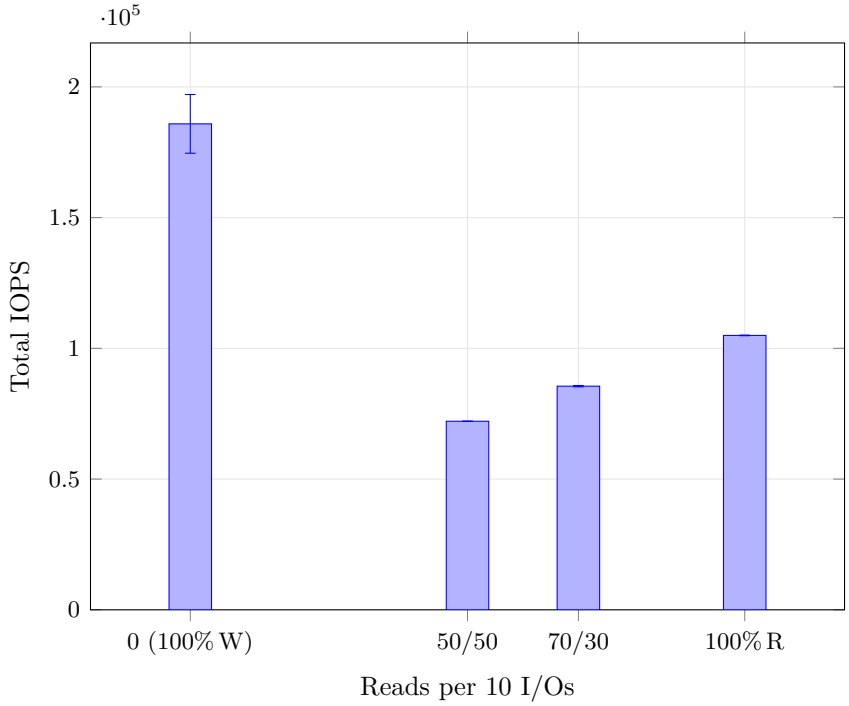


Figure 2: Total 4 KiB random IOPS at QD=16 across read/write mixes. Both pure workloads beat the mixes; the 50/50 mix is the worst.

Table 3: Read/write mix at QD=16, 4 KiB random (medians).

Mix	Total IOPS	Mean lat (µs)	p99 (µs)
100% read	104 945	151.5	458
70/30	85 535	185.8	707
50/50	72 119	221.0	995
100% write	185 860	85.3	857

Fixed	4 KiB random; iodepth 16; one job; `O_DIRECT`
Varied	reads per 10 I/Os: 10 (100% R), 7 (70/30), 5 (50/50), 0 (100% W)
Script	`SSD_Performance_Profiling/scripts/run.sh rw_mix`

6.1 Analysis

The mix sweep confirms the hypothesis and then some. Pure writes are fastest (185 860 IOPS, 85 µs mean) because at QD=16 the controller coalesces them into its SLC/DRAM buffer; pure reads reach 104 945 IOPS. But mixing is strictly worse than either pure case: the 70/30 mix drops to 85 535 and the 50/50 mix to 72 119 IOPS—below even the read-only number—because every write that lands mid-stream stalls the flash channel a read then queues behind, and read-modify interference defeats the write buffer's coalescing. Latency tells the same story from the tail: p99 climbs monotonically with the write fraction, from 458 µs (pure read) through 707 and 995 µs for the mixes, as background program/erase operations inject outliers. The lesson is that a "70% read" workload is not 0.7 of the read result plus 0.3 of the write result; the interaction term is large and negative, which is exactly why the spec requires measuring the mixes rather than interpolating them.

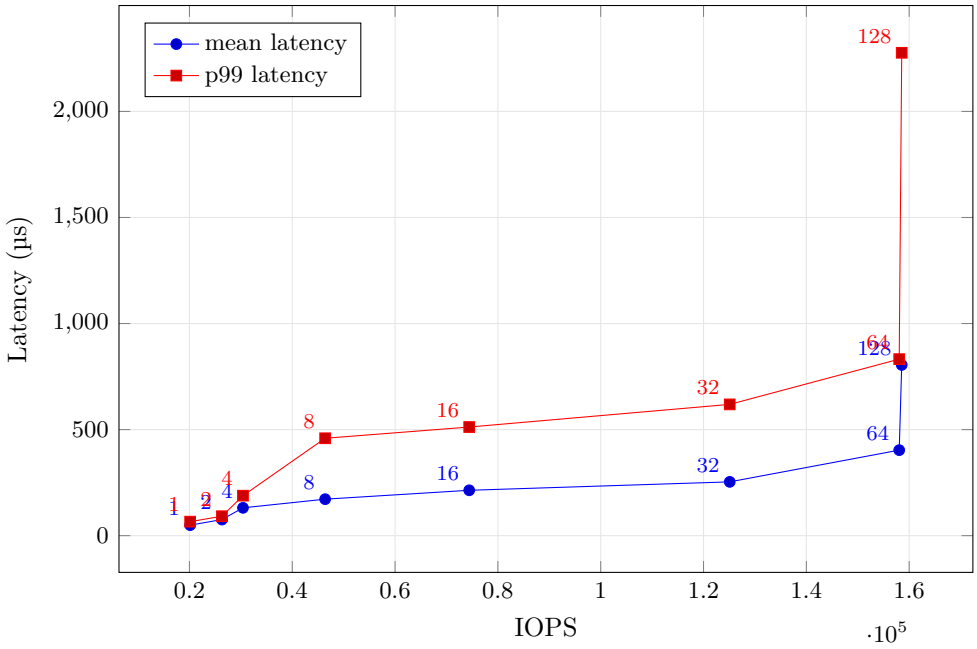


Figure 3: Throughput vs. latency; each point is labelled with its queue depth. IOPS saturates near 158 k at QD=64; past the knee, QD=128 doubles latency for no extra throughput.

7 Queue-Depth Sweep

Hypothesis (written before measuring)

IOPS should rise steeply with queue depth as the SSD's parallel NAND channels fill, then saturate at a knee beyond which latency grows roughly linearly with queue depth for no extra throughput. By Little's Law the number of outstanding I/Os $L = \lambda W$ should track the configured iodepth while the queue stays full; where it stops tracking reveals a host-side or engine limit.

Fixed	4 KiB random read; one job; `O_DIRECT`
Varied	iodepth 1 to 128
Effective QD	verified in-band via the Little's Law table below (no `iostat` available)
Script	`SSD_Performance_Profiling/scripts/run.sh queue_depth`

7.1 Little's Law check

With the queue kept full the outstanding-I/O count should satisfy $L = \lambda W$ (IOPS $\times$ mean latency) and equal the configured iodepth [2].

7.2 Analysis

IOPS climbs almost linearly through QD=16 (20k $\rightarrow$ 74k), begins to bend at QD=32 (125k) and saturates at QD=64 (158k); QD=128 adds 0.3% throughput while mean latency doubles from 403 to 805 µs. The knee is therefore at QD $\approx$ 64, where the device's channel parallelism is fully engaged. The Little's Law column is strikingly clean: λW equals the configured iodepth to within a percent at *every* depth including QD=128 ($L = 127.6$), so the `libaio` engine really does keep the queue full—the effective queue depth matches the requested one, which is the in-band substitute for an

Table 4: Little's Law: measured λW against the configured queue depth (4 KiB random read).

iodepth	λ (IOPS)	W (µs)	$L = \lambda W$	
1	20 137	49.6	1.00	
2	26 325	75.9	2.00	
4	30 416	131.3	3.99	
8	46 398	172.1	7.99	
16	74 420	213.9	15.92	
32	125 097	253.7	31.74	knee region
64	158 064	403.2	63.73	saturation
128	158 551	804.8	127.6	latency-only growth

`iostat` aqu-sz check. This is also the textbook signature of saturation: past the knee λ is flat, so the only way L can keep growing with iodepth is for W to grow proportionally—exactly the linear latency rise we see. The practical reading: QD $\approx$ 32–64 extracts essentially all the throughput at a quarter-millisecond latency; deeper queues only buy latency. Here the saturation is the device's, not the harness's, since Little's Law holds all the way out.

8 Tail Latency

Hypothesis (written before measuring)

Moving from a mid queue depth to the near-knee depth should raise the median modestly but stretch the upper percentiles much more: deeper queues mean more requests waiting behind the occasional slow flash operation, so the p99/p99.9 gap over p50 should widen.

Fixed	4 KiB random read; one job; 6 s runs, 3 reps
Varied	iodepth 8 (mid) and 32 (near-knee, from Section 7)
Script	`SSD_Performance_Profiling/scripts/run.sh tail`

8.1 Analysis

At QD=8 the percentiles are p50 170, p95 281, p99 460, p99.9 635 µs (46 k IOPS); at QD=32 they are p50 255, p95 396, p99 563, p99.9 808 µs (126 k IOPS). The median rises only 1.5× while IOPS nearly triples, but the whole distribution shifts up and the p99.9/p50 ratio stays around 3–3.7×: the tail is dominated by requests that arrive while the controller is busy with a background program/erase/GC operation, and a deeper queue makes a waiting request more likely. These tails are mild compared with a rotational disk (no seeks), but they still matter: at QD=32 one read in a thousand takes ∼800 µs, 3× the median, which for a latency-SLA service is the number that governs provisioning, not the mean. This is why the knee from Section 7 is a latency decision as much as a throughput one—running at QD=64+ buys little IOPS but inflates exactly these tails.

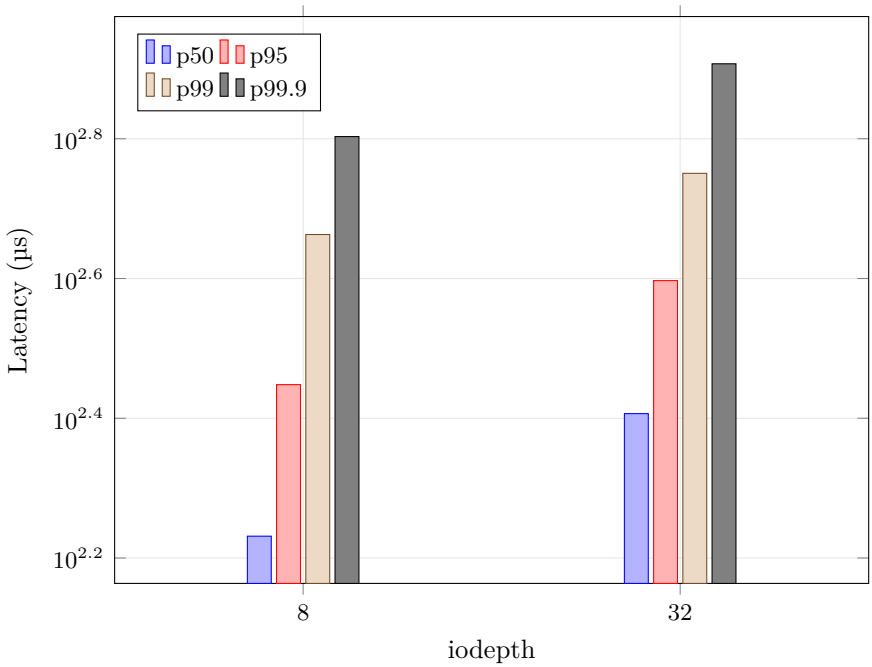


Figure 4: Latency percentiles at a mid (QD=8) and near-knee (QD=32) queue depth. The median barely moves but the upper tail stretches with queueing.

9 Device State and Working Set

Hypothesis (written before measuring)

Back-to-back sustained sequential writes should first ride the SLC/DRAM write cache (a burst), then settle to a lower steady-state throughput once the cache must flush to QLC in the background. Occasional latency spikes from garbage collection should appear in the upper tail even while mean throughput is steady.

Workload	128 KiB sequential write, iodepth 4, eight back-to-back 2 s intervals
LBA range	the 2 GiB disposable file (a small fraction of the 512 GB device)
State	preconditioned (full sequential write pass) before the intervals
Script	`SSD_Performance_Profiling/scripts/run.sh device_state`

9.1 Analysis

Throughput is not a clean monotonic SLC cliff at this file size: the first interval delivers 959 MiB/s, dips to 631 and 776 MiB/s in intervals 1–2 (the controller flushing buffered data to QLC), then recovers to a steady ∼1060 MiB/s for the remaining five intervals. Because the working set is only 2 GiB of a 512 GB drive and the run is short, we do not drive the SLC cache to exhaustion—so there is no dramatic steady-state collapse, only the initial flush transient. The tail tells the GC story the mean hides: p99.9 latency sits at 17–19 ms in every interval even while mean throughput is flat, i.e. periodic background flush/garbage-collection pauses punctuate otherwise-fast writes. This is the honest device-state picture for a small-footprint run; a full-capacity, minutes-long write would be needed to measure the true SLC-cache size and steady-state QLC write rate, which we did not do (the disposable file is deliberately small).

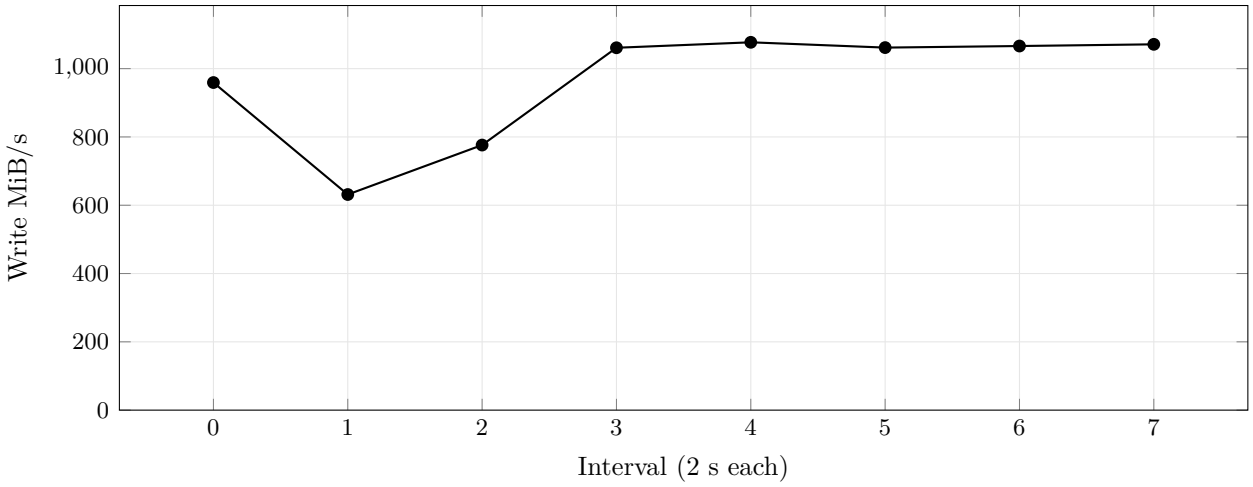


Figure 5: Sustained 128 KiB random-free sequential-write throughput over eight 2 s intervals. Throughput dips early (cache flush) then holds a steady ∼1060 MiB/s.

Table 5: Same 4 KiB sequential-read workload, direct vs. buffered (medians). The only difference is the `O_DIRECT` flag.

Setup	IOPS	MiB/s	Mean (µs)	p99 (µs)
`O_DIRECT` (correct)	86 201	337	11.57	21.3
buffered (page cache)	2 057 063	8035	0.45	0.91

9.2 SMART / thermal

SMART attributes (temperature, data-units-written, media errors) and NVMe health logs require root on this host—`smartctl -i /dev/nvme0n1` returns *Permission denied*—so we cannot report before/after temperature or wear counters. No thermal throttling was inferred from the throughput trace (it recovers and holds rather than decaying), but we cannot confirm this from the controller directly; this is recorded as a limitation (Section 11).

10 Misleading Setup

Hypothesis (written before measuring)

Dropping `O_DIRECT` lets the OS page cache serve repeated reads of the same region from RAM after the first pass, so a buffered benchmark will report throughput and latency that describe DRAM, not the SSD—orders of magnitude "better" than the real device, and entirely fictitious as a storage result.

Pitfall	Buffered I/O measuring the page cache instead of the device
Wrong setup	4 KiB sequential reads over a 64 MiB region, *no* `O_DIRECT`
Right setup	Identical workload *with* `O_DIRECT` (page cache bypassed)
Script	`SSD_Performance_Profiling/scripts/run.sh pitfall`

10.1 Explanation

The buffered run reports 2.06 million IOPS at 8.0 GiB/s and sub-microsecond latency—a 24× inflation over the correct 86 k IOPS / 337 MiB/s figure. None of it is a storage measurement: after the first pass the 64 MiB region lives entirely in the page cache, so every subsequent read is a memcpy from RAM. 8 GiB/s is DRAM bandwidth, and 0.45 µs is a cache-hit memory latency, not an NVMe access. This is the single easiest way to publish a fraudulently fast SSD number, and an AI-generated benchmark falls into it readily because omitting `O_DIRECT` is the *default* behaviour of `open()`. Three cheap detectors flag it: (1) the throughput exceeds the device's interface ceiling and even its own QD=1/QD=128 direct numbers by more than an order of magnitude; (2) sub-microsecond "storage" latency is physically impossible for NAND flash; and (3) the number scales with the *region size* (shrinks once the region exceeds RAM), which a true device result must not. Our methodology avoids it structurally: `O_DIRECT` is the default for every real run (the buffered path exists only to produce this comparison), and the validation step (Section 3) asserts the direct-vs-buffered gap to prove the cache is bypassed. The same trap underlies "wrong effective queue depth": if the async engine never keeps iodepth requests in flight, a QD=32 label can really be a QD=2 result—which is why we verify the effective depth with the Little's Law table (Table 4) rather than trusting the requested value.

11 Anomalies and Limitations

11.1 Unexpected results

- **Random writes beat random reads at low queue depth.** At QD=1, 4 KiB writes hit 68 k IOPS (14.7 µs) versus reads at 20 k (49.6 µs), and at QD=16 pure writes (186 k IOPS) beat pure reads (105 k). Hypothesis: the controller acknowledges writes from its SLC/DRAM buffer before the flash program completes, whereas a read must wait for the NAND sense. Consistent with the write buffer also explaining why *mixed* workloads are slower than either pure case (Section 6).

- **Sequential QD=1 read throughput is modest (**227 MiB/s**).** Lower than the drive's headline rating. Hypothesis: with one outstanding 128 KiB request and `O_DIRECT` (no OS read-ahead), the device is under-fed; the block-size and queue-depth sweeps show throughput rising to 1.5 GiB/s with larger blocks and 620 MiB/s of random 4 KiB at QD=128, so this is a single-request artifact, not a device ceiling.

- **Sustained-write throughput dips then recovers** rather than falling off an SLC cliff (Figure 5). Hypothesis: the 2 GiB footprint is too small and the run too short to exhaust the SLC cache; the dip is the initial flush transient. Tested only indirectly—a full-capacity run would be needed to see the true steady-state QLC rate.

11.2 Limitations

- **FIO not available.** not available (not installed); measurements use `blkbench`, an O_DIRECT `libaio` stand-in. FIO-only conveniences—full latency histograms beyond p99.9, `numjobs` scaling, built-in SMART logging and its exact ioengine knobs—are approximated or omitted, not faked.

- **No `iostat/perf` cross-check.** Effective queue depth is verified in-band via Little's Law (Table 4) rather than against `iostat` aqu-sz.

- **SMART/thermal need root.** Temperature, data-units-written and media-error counters are unavailable (`smartctl`/`nvme` require privileges we lack), so thermal/throttling and wear claims are limited to what the throughput trace shows.
- **Filesystem, not raw device; small LBA range.** I/O goes to a 2 GiB disposable file on an ext4 filesystem (the data-safety requirement), not a raw partition, so ext4 extent mapping adds a thin layer and only 0.4% of the 512 GB LBA space is exercised. Full-span and true steady-state behaviour are out of scope.
- **Fixed data pattern; short runs.** Writes use a constant `0xA5` fill (compressibility not varied) and runs are 4 s to 6 s; the SLC-cache size and long-term steady state were not measured. One drive, one firmware.
- **Report build backend.** Builds cleanly with `make ssd` and `make final`; the host has `bibtex` but not `biber`, so `biblatex` uses `backend=bibtex`. The only artifact is a benign "Please (re)run BibTeX" notice—no unresolved `[?]` markers.

12 AI-Use Note

Summarized from `AI_USE_LOG.md` in the code repository.

Tools used

Claude Code (Claude Opus 5) for coding, debugging and drafting this report; GCC gcc (GCC) 16.2.1 20260810 for the builds. The AI wrote code and prose; every number comes from real runs on the machine in Table 1.

What they helped generate

`blkbench.c` (the O_DIRECT `libaio` benchmark standing in for FIO), the data-safety gate (`safety.sh`), the experiment driver (`run.sh`) and its CSV reduction, and the first drafts of these sections.

What was independently modified, tested, or rejected

Three concrete corrections. (1) *The wrong device was being tested.* The scratch directory defaulted to the in-repo `data/scratch/`, which lives on the box's 2 TB SATA *HDD* (`rotational`=1), so the first full dataset showed QD=1 random latencies in the *milliseconds*—textbook rotational behaviour, not an SSD. Reading the `env.txt` `rotational` flag caught it; the scratch path was made overridable (`ACS_SCRATCH`) and repointed at the Micron NVMe SSD mounted at `/`, after which QD=1 random read dropped to ∼50 µs. The AI's original claim that "the NVMe is read-only to us" was simply false and was disproved by a one-line write test. (2) An early version measured *submission* latency (time to enqueue) rather than *completion* latency; it was changed to time each I/O to completion, which is what the percentiles must describe. (3) A convenience default left the page cache enabled, reporting ∼2 million IOPS; rather than quote it, it became the required pitfall (Section 10), and `O_DIRECT` was made the default for every real run.

Validation

The safety gate is unit-tested in `validate.sh` (it must refuse `/dev/nvme0n1`, `/etc/passwd` and `/`). Direct-vs-buffered I/O differs by two orders of magnitude, proving the page cache is bypassed; sequential throughput stays under the interface ceiling; and the Little's-Law table (Table 4) is the in-band effective-queue-depth check in place of an `iostat`/`perf` cross-check (neither available here). Every figure traces to a committed raw/processed file (Section A).

13 Conclusion

The Micron 2210 NVMe SSD shows the queueing-driven throughput–latency tradeoff the project set out to measure, and the numbers are unambiguous once the device is correctly targeted. At QD=1, 4 KiB random reads cost 49.6 µs (20 k IOPS) and writes only 14.7 µs (68 k IOPS), because the controller's write buffer hides the flash program latency. Block size moves the bottleneck from IOPS-limited to throughput-limited—sequential reads saturate near 1.5 GiB/s—and mixing reads with writes is strictly worse than either pure workload (the 50/50 mix drops to 72 k IOPS, below the read-only 105 k). The queue-depth sweep locates the knee at QD $\approx$ 64, where IOPS saturates at 158 k; beyond it, latency grows linearly for no throughput, and Little's Law holds to within a percent at every depth—so the effective queue depth is real and the saturation is the device's, not the benchmark's. Tails are mild (no seeks) but p99.9 still runs 3$\times$ the median and is the number a latency SLA must budget for. The sharpest lesson is methodological: the same workload *without* `O_DIRECT` reported 2 million IOPS of pure fiction (the page cache), and the first full dataset was accidentally taken against the machine's HDD until the `rotational` flag gave it away. Storage, like memory, is governed by queueing—but only a measurement that bypasses the cache, verifies its effective queue depth, and confirms which device it is even talking to can be trusted to say so.

References

[1] J. Axboe, *Fio: Flexible I/O tester*, Referenced as the standard tool; not installed on the test host, so a libaio stand-in (blkbench) was used. [Online]. Available: `https://github.com/axboe/fio`

[2] J. D. C. Little, "A proof for the queuing formula: $L = \lambda W$," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961. DOI: `10.1287/opre.9.3.383`

A Reproducibility

Code, scripts and raw data: `github.com/SKempiak/ACS-Code`, on the `class-wide-project` submission branch (the exact commit is recorded in `git log`).

```
git clone https://github.com/SKempiak/ACS-Code.git && cd ACS-Code
git checkout <commit>
make tools                                  # tools/summarize
make -C SSD_Performance_Profiling           # build/blkbench (O_DIRECT, libaio)
make -C SSD_Performance_Profiling test      # safety + physical-ordering sanity checks
scripts/collect_env.sh --tex > ../ACS-Latex/common/system_info.tex
# Point the disposable-file scratch at the device you want to profile (here the NVMe SSD at /):
ACS_SCRATCH="$HOME/acs_ssd_scratch" SSD_Performance_Profiling/scripts/run.sh all
scripts/sync_data.sh SSD_Performance_Profiling
```

The `ACS_SCRATCH` override selects the device under test; the `safety.sh` gate is identical wherever it points (a plain disposable file inside the scratch directory, never a device or mountpoint). Omitting it profiles the in-repo `data/scratch/` filesystem instead.

Each raw directory holds `raw.csv` (one row per repetition, with full configuration metadata) and `env.txt` (the `collect_env.sh` snapshot plus the device model and `rotational` flag); the processed CSVs plotted above come from `tools/summarize` and are copied into this report's `data/` by `scripts/sync_data.sh`. Every number quoted traces to one of these files.

Table 6: Raw data behind each figure.

Figure / table	Raw run directory (`SSD_Performance_Profiling/data/raw/...`)
Table 2	`zero_queue/20261020-001056`
Figure 1	`block_size/20261020-001056`
Figure 2	`rw_mix/20261020-001056`
Figure 3, Table 4	`queue_depth/20261020-001056`
Figure 4	`tail/20261020-001056`
Figure 5	`device_state/20261020-001056`
Table 5	`pitfall/20261020-001056`